

DOCUMENTO DE ARQUITECTURA DE SOFTWARE (SAD)

Modelo de Vistas 4+1 (IEEE Std 1471 / ISO 42010) — Patrón Clean Architecture + Feature-First

LÍDER DE ARQUITECTURA & QA Alan Martinez	FECHA Y VERSIÓN 2026-08-25 Versión 2.0 (Arquitectura Consolidada)	EQUIPO DE DESARROLLO I. Cardozo, F. Sarraute, A. Heredia, M. Silvani	ESTÁNDAR & PATRÓN Modelo 4+1 · Clean Arq + Feature-First
---	---	--	---

1. VISIÓN GENERAL DE LA ARQUITECTURA Y SELECCIÓN DEL PATRÓN

El **Documento de Arquitectura de Software (SAD)** formaliza la estructura de la Plataforma Hospitalaria de Código Azul mediante el **Modelo de Vistas 4+1 de Philippe Kruchten** (IEEE Std 1471 / ISO 42010). Se adopta el patrón **Clean Architecture adaptado a Feature-First con Repository Pattern**, desacoplando el dominio clínico de los detalles de infraestructura.

DECISIÓN ARQUITECTÓNICA CLAVE (ADR-01)

La arquitectura Feature-First divide el backend en módulos funcionales autónomos (auth, codigo_azul, fcm). Esto permitió el desarrollo concurrente sin conflictos en Git: Ivan Cardozo en Backend Core & DB, Alex Heredia en FCM & Telemetría, y Franco Sarraute en Frontend & Mobile.

2. VISTAS DE LA ARQUITECTURA (MODELO 4+1)

2.1. Vista Lógica (Capas Concéntricas)

- Capa 1 (Domain / Core):** Entidades del dominio clínico, máquinas de estado FSM y tipos ENUM nativos de PostgreSQL (estado_incidente, rol_usuario).
- Capa 2 (Use Cases / Application):** Lógica de negocio aislada (ActivarCodigoAzul, ConfirmarACK, CancelarIncidente, ResolverIncidente, RegistrarTokenFCM, DespacharPush).
- Capa 3 (Data / Repository):** Abstracción de persistencia relacional SQL sobre pg Pool con soporte para transacciones ACID (BEGIN / COMMIT / ROLLBACK).
- Capa 4 (Presentation / Drivers):** Controladores Express 5, Middlewares de autenticación JWT y RBAC, Gateway de WebSockets (Socket.IO) y SDK de Firebase Admin.

2.2. Vista de Procesos (Flujo Concurrente y Tiempo Real)

Ante una solicitud POST /api/v1/incidentes/activar:

- El middleware valida el token JWT y los claims del rol (MEDICO_ACTIVADOR / ADMINISTRADOR).
- El repositorio consulta la **Barrera de Idempotencia de 60s** sobre el índice espacial de la base de datos.
- Se ejecuta una transacción ACID en PostgreSQL 18 insertando el incidente y el evento de auditoría en microsegundos.
- El bus desacoplado appEvents despacha en paralelo: a) Broadcast por Socket.IO al Dashboard Web (≤ 35 ms) y b) Push Notification de alta prioridad vía Firebase Admin SDK a los terminales Android.

2.3. Vista de Desarrollo (Estructura Modular Feature-First)

```
src/
├── app.js                # Configuración de Express 5, Helmet, CORS y Error Handler
├── server.js            # Servidor HTTP y exportación para Socket.IO
├── core/                # Núcleo transversal compartido
│   ├── config/          # db.js (Pool pg ACID), env.js, firebase.config.js
│   ├── events/          # Bus de eventos EventEmitter desacoplado
│   ├── helpers/         # api-error.js, api-response.js, async-handler.js
│   ├── middlewares/     # auth.middleware (JWT), rbac.middleware, error-handler
│   └── sockets/         # socket.gateway.js (Gateway de alertas en tiempo real)
├── features/           # Módulos funcionales (Feature-First)
│   ├── auth/           # Repositorios, use-cases (login) y rutas /auth
│   ├── codigo_azul/    # Domain (FSM), repositorios (ACID/Auditoría), use-cases y rutas
│   └── fcm/            # Repositorio de tokens, use-cases (push/telemetría) y rutas /fcm
```

2.4. Vista Física y de Despliegue (Modelo C4 / Nodos)

El ecosistema se despliega en contenedores Docker orquestados mediante Docker Compose:

- **Nodo 1 (Cliente Web Dashboard):** Single Page Application en React 18 / Vite servida en puerto 5173 / 3000.
- **Nodo 2 (Cliente Móvil PWA):** Terminales táctiles Android con Service Worker, Web Audio API y cliente FCM.
- **Nodo 3 (Backend Core):** Node.js v24 LTS / v20+ ejecutando Express 5.2.1 y Socket.IO 4.8.
- **Nodo 4 (Database Relacional):** PostgreSQL 18.4 Alpine con persistencia ACID, extensiones pgcrypto y triggers plpgsql.
- **Nodo 5 (Cloud Provider):** Firebase Cloud Messaging (FCM HTTP v1) para notificaciones push en segundo plano.

3. MATRIZ DEL STACK TECNOLÓGICO DEFINITIVO

Tabla 1. Stack Tecnológico Oficial y Justificación Operativa

CAPA ARQUITECTÓNICA	TECNOLOGÍA SELECCIONADA	JUSTIFICACIÓN TÉCNICA Y CUMPLIMIENTO
Backend Core & REST	Node.js (v24 LTS) + Express 5.2.1	Event Loop no bloqueante ideal para I/O intensivo; manejo nativo de promesas async/await y alta concurrencia.
Tiempo Real (WebSockets)	Socket.IO v4.8.3	Canal full-duplex de ultra-baja latencia (15-35 ms) con reconexión automática y heartbeat activo.
Notificaciones Push	Firebase Admin SDK v13.4.0 (FCM)	Despacho de alertas sonoras prioritarias con bypass de Doze Mode en terminales Android bloqueados.
Base de Datos Relacional	PostgreSQL 18.4 (Driver nativo pg)	Transaccionalidad ACID plena, tipos ENUM, JSONB para telemetría y trigger inmutable médico-legal.
Frontend Web & Mobile	React 18 + Vite 5 + PWA	Renderizado reactivo instantáneo, Web Audio API para sirenas acústicas y caché offline con Service Worker.

4. REFERENCIAS NORMATIVAS (APA 7.ª EDICIÓN)

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley Professional.

IEEE Computer Society. (2000). *IEEE Recommended Practice for Architectural Description of Software-Intensive Systems* (IEEE Std 1471-2000 / ISO/IEC 42010). IEEE.

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design* (1.ª ed.). Prentice Hall.